ONEDRAFT

CAD-AI — Aria Powered

Project Review, Redline, Acceptance & Approval Engine

Version 0.1

Status: Implementation Specification
Subsystem: Review / Redline / Acceptance / Approval
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered

---

1. SUBSYSTEM PURPOSE

The OneDraft Project Review, Redline, Acceptance & Approval Engine is the controlled human-review and project-finalization subsystem of OneDraft.

It receives generated documentation from the Documentation & Drawing Generation Engine and provides the mechanisms required to:

review

mark up

redline

request changes

evaluate changes

resolve redlines

regenerate affected documentation

validate the resulting revision

submit for customer review

record customer acceptance

route for professional review

track municipal approval

finalize a project package

preserve the resulting project record

The subsystem does not replace the underlying project model.

It operates against the existing:

```
PROJECT
 ↓
MODEL
 ↓
GEOMETRY
 ↓
COMPLIANCE
 ↓
VALIDATION
 ↓
REVISION
 ↓
DOCUMENTATION
```

and establishes the controlled review path:

```
DOCUMENTATION
 ↓
REVIEW
 ↓
REDLINE
 ↓
MODEL CHANGE
 ↓
VALIDATION
 ↓
DOCUMENT REGENERATION
 ↓
REVIEW
 ↓
ACCEPTANCE
 ↓
PROFESSIONAL REVIEW
 ↓
APPROVAL
 ↓
FINALIZATION
```

---

2. ARCHITECTURAL PRINCIPLE

OneDraft shall distinguish between:

generated

reviewed

accepted

professionally reviewed

approved

and:

finalized

These states are not interchangeable.

Generation means the system produced documentation.

Review means a human or authorized process examined the documentation.

Acceptance means the designated customer accepted the identified package.

Professional review means an appropriately designated professional reviewed the specified scope.

Municipal approval means an external authority has recorded a decision.

Finalization means OneDraft has locked the identified project state and associated document package as a controlled project deliverable.

---

3. REVIEW SYSTEM OF RECORD

The Review Engine shall never become a competing project model.

The authoritative chain remains:

```
PROJECT
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
 ↓
VALIDATION RUN
 ↓
DOCUMENTATION
 ↓
REVIEW RECORD
 ↓
ACCEPTANCE / APPROVAL
```

Review records describe decisions concerning an existing project state.

They do not independently redefine that state.

---

4. REVIEW OBJECTIVES

The subsystem shall allow reviewers to determine whether documentation:

corresponds to the intended model

contains requested design changes

contains unresolved issues

requires correction

requires additional information

is suitable for customer acceptance

is suitable for professional review

is ready for finalization

Review findings shall be traceable to the specific revision and model version being reviewed.

---

5. REVIEW SESSIONS

A review session represents a controlled examination of a project revision.

Conceptually:

```
review_session
    ↓
review_items
    ↓
redlines
    ↓
resolution
    ↓
validation
    ↓
review_completion
```

A review session must identify:

```
project_id
revision_id
model_version_id
document_package_id
reviewer_id
review_type
status
created_at
completed_at
```

---

6. REVIEW TYPES

Initial review types:

```
INTERNAL_DESIGN_REVIEW
CUSTOMER_REVIEW
COORDINATION_REVIEW
CODE_REVIEW
DOCUMENT_REVIEW
PROFESSIONAL_REVIEW
FINAL_REVIEW
```

Future review types may be added without changing the underlying project model.

---

7. REVIEW SESSION TABLE

```
revisions.review_sessions
```

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
document_package_id UUID
reviewer_id UUID NOT NULL
review_type VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
summary TEXT
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_review_sessions_project
idx_review_sessions_revision
idx_review_sessions_reviewer
idx_review_sessions_status
```

---

8. REVIEW SESSION STATES

Initial states:

```
CREATED
IN_PROGRESS
CHANGES_REQUESTED
READY_FOR_REVIEW
```

```
COMPLETED
CANCELLED
SUPERSEDED
```

A completed review must identify its resulting disposition.

---

9. REVIEW DISPOSITIONS

Initial dispositions:

```
ACCEPTED
ACCEPTED_WITH_NOTES
CHANGES_REQUIRED
REJECTED
DEFERRED
ESCALATED
```

A review cannot become `COMPLETED` without a disposition.

---

10. REVIEW ITEMS

Individual findings shall be stored independently from the review session.

```
revisions.review_items
```

Fields:

```
id UUID PRIMARY KEY
review_session_id UUID NOT NULL
project_id UUID NOT NULL
revision_id UUID NOT NULL
drawing_id UUID
sheet_id UUID
view_id UUID
target_object_id UUID
category VARCHAR(64) NOT NULL
severity VARCHAR(32) NOT NULL
description TEXT NOT NULL
instruction TEXT
status VARCHAR(32) NOT NULL
created_by UUID NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

11. REVIEW ITEM CATEGORIES

Initial categories:

```
DESIGN
GEOMETRY
CODE
DIMENSION
ANNOTATION
COORDINATION
DOCUMENTATION
SCHEDULE
GRAPHICS
CLIENT_REQUEST
CONSTRUCTABILITY
OTHER
```

---

12. REVIEW ITEM SEVERITY

Initial severity:

```
INFO
MINOR
MAJOR
CRITICAL
```

Severity shall not itself determine whether a project can be finalized.

Finalization rules shall evaluate unresolved mandatory findings and applicable workflow requirements.

---

13. REVIEW ITEM STATUS

Initial statuses:

```
OPEN
ACKNOWLEDGED
REDLINE_CREATED
IN_PROGRESS
RESOLVED
REJECTED
DEFERRED
SUPERSEDED
```

---

14. REDLINE ENGINE

The Redline Engine is the controlled mechanism for converting review findings into actionable model changes.

The existing redline architecture remains authoritative.

The Review Engine extends it by associating redlines with:

```
review_session
review_item
revision
drawing
sheet
target_object
resolution
```

---

15. REDLINE LIFECYCLE

The standard redline lifecycle becomes:

```
OPEN
 ↓
TRIAGED
 ↓
ACCEPTED_FOR_CHANGE
 ↓
IMPLEMENTATION
 ↓
VALIDATION
 ↓
RESOLVED
```

Alternative terminal states:

```
REJECTED
DEFERRED
SUPERSEDED
```

---

16. REDLINE CREATION

A reviewer may create a redline from:

a drawing

a sheet

a view

a model object

a review item

a spatial markup

A redline must identify the revision against which it was created.

---

17. REDLINE IMMUTABILITY

Once a redline has been submitted for implementation, its original instruction shall remain immutable.

If the instruction changes, OneDraft creates a new redline or superseding record.

This preserves review provenance.

---

18. REDLINE MARKUP

Markup geometry may contain:

```
point
line
polyline
rectangle
circle
polygon
cloud
arrow
freehand
text_anchor
```

The geometry is review metadata.

It does not automatically become project geometry.

---

19. REDLINE INSTRUCTION

Every actionable redline must contain a human-readable instruction.

Example:

```
Move rear bedroom wall 300 mm north while maintaining
the exterior envelope and required clearances.
```

The instruction becomes input to the Aria command boundary.

---

20. ARIA REVIEW ASSISTANCE

Aria may analyze a review item or redline and propose:

```
affected_objects
operation
parameters
constraints
expected_impacts
validation_requirements
```

Aria shall not bypass the existing command boundary.

The sequence remains:

```
REVIEW
 ↓
REDLINE
 ↓
ARIA ANALYSIS
 ↓
COMMAND
 ↓
MODEL MUTATION
 ↓
VALIDATION
```

---

21. REVIEW-TO-COMMAND BOUNDARY

A redline shall never directly modify the project model.

Instead:

```
REDLINE
 ↓
COMMAND REQUEST
 ↓
COMMAND VALIDATION
 ↓
MODEL TRANSACTION
 ↓
NEW REVISION
```

This preserves the existing revision architecture.

---

22. CHANGE REQUESTS

The subsystem shall support structured change requests.

```
revisions.change_requests
```

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
review_session_id UUID
revision_id UUID NOT NULL
source_type VARCHAR(64) NOT NULL
source_id UUID
title VARCHAR(255) NOT NULL
description TEXT NOT NULL
priority VARCHAR(32)
status VARCHAR(32) NOT NULL
requested_by UUID NOT NULL
assigned_to UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

23. CHANGE REQUEST STATES

```
OPEN
ASSIGNED
ANALYZING
IMPLEMENTING
VALIDATING
READY_FOR_REVIEW
COMPLETED
REJECTED
DEFERRED
CANCELLED
```

---

24. CHANGE REQUEST ASSIGNMENT

Change requests may be assigned to:

```
ARIA
SYSTEM
DESIGNER
DRAFTER
REVIEWER
PROFESSIONAL
ADMINISTRATOR
```

Assignment does not grant permissions beyond the user's or service's existing authorization scope.

---

25. REDLINE RESOLUTION

A redline becomes resolved only when:

```
requested change evaluated
 ↓
model command executed
 ↓
new revision created
 ↓
affected geometry updated
 ↓
affected documentation regenerated
 ↓
required validation completed
 ↓
resolution recorded
```

A textual acknowledgement alone does not constitute implementation.

---

26. RESOLUTION RECORD

The existing:

```
revisions.redline_resolutions
```

record shall be extended conceptually to include:

```
review_item_id
change_request_id
result
implemented_revision_id
validation_run_id
affected_documents
resolution_type
```

---

27. RESOLUTION TYPES

Initial types:

```
IMPLEMENTED
NOT_APPLICABLE
REJECTED
DEFERRED
DUPLICATE
SUPERSEDED
```

---

28. AUTOMATIC IMPACT ANALYSIS

When a model change resolves a redline, OneDraft shall identify affected:

```
elements
spaces
levels
assemblies
constraints
views
drawings
sheets
schedules
validation rules
document packages
```

This information shall be derived from the existing project dependency graph.

---

29. DOCUMENT IMPACT GRAPH

The subsystem shall use the existing dependency relationships to determine whether a change affects documentation.

Conceptually:

```
ELEMENT
  ↓
VIEW
  ↓
DRAWING
  ↓
SHEET
  ↓
DOCUMENT PACKAGE
```

A model change shall not require regeneration of unrelated documentation when dependency analysis demonstrates that it is unaffected.

---

30. REVIEW PACKAGE

A review package is the controlled set of documentation submitted for examination.

```
revisions.review_packages
```

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
document_package_id UUID NOT NULL
review_session_id UUID
package_hash VARCHAR(128)
status VARCHAR(32) NOT NULL
```

```
created_at TIMESTAMPTZ NOT NULL
```

---

31. REVIEW PACKAGE INTEGRITY

Every review package shall identify:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
document_package_id
package_hash
```

This prevents a reviewer from unknowingly reviewing a document package generated from a different project state.

---

32. PACKAGE LOCK

Once a review package is issued:

```
review_package.status = ISSUED
```

the associated document package shall remain immutable.

Changes create a new revision and a new review package.

---

33. CUSTOMER REVIEW

Customer review shall be represented as a controlled workflow.

Conceptually:

```
GENERATED
 ↓
INTERNAL REVIEW
 ↓
CUSTOMER REVIEW
 ↓
CUSTOMER COMMENTS
 ↓
REDLINES
 ↓
REVISION
 ↓
CUSTOMER REVIEW
```

34. CUSTOMER REVIEW RECORD

`revisions.customer_reviews`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
review_package_id UUID NOT NULL
customer_id UUID NOT NULL
status VARCHAR(32) NOT NULL
comments TEXT
submitted_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

35. CUSTOMER REVIEW STATES

```
NOT_STARTED
ISSUED
IN_REVIEW
COMMENTS_SUBMITTED
ACCEPTED
CHANGES_REQUESTED
EXPIRED
CANCELLED
```

---

36. CUSTOMER ACCEPTANCE

Customer acceptance shall remain separate from professional review and municipal approval.

Acceptance means the designated customer has accepted the identified project package for the defined scope.

It does not automatically mean:

```
professional approval
municipal approval
permit approval
construction approval
```

---

37. ACCEPTANCE RECORD

The existing:

```
revisions.acceptance_records
```

shall serve as the canonical acceptance record.

It shall identify:

```
project_id
revision_id
model_version_id
document_package_id
code_manifest_id
customer_id
acceptance_status
accepted_at
acknowledgements
```

---

38. ACCEPTANCE STATES

Initial states:

```
PENDING
ACCEPTED
ACCEPTED_WITH_NOTES
REJECTED
WITHDRAWN
SUPERSEDED
```

---

39. ACCEPTANCE ACKNOWLEDGEMENTS

The acceptance record may contain structured acknowledgements such as:

```
reviewed_documents
reviewed_revision
reviewed_scope
known_exceptions
customer_comments
```

The system shall preserve the exact accepted revision.

---

40. ACCEPTANCE REVISION LOCK

Acceptance is revision-specific.

If the project model changes after acceptance:

```
accepted_revision != current_revision
```

and the previous acceptance shall not automatically apply to the new revision.

The system shall identify the acceptance as associated with the prior revision.

---

41. POST-ACCEPTANCE CHANGE

Any substantive change after customer acceptance shall produce:

```
new revision
new validation
new documentation
new review package
new acceptance decision
```

where required by workflow.

---

42. PROFESSIONAL REVIEW

Professional review shall remain distinct from AI review and customer acceptance.

The existing:

```
revisions.professional_reviews
```

table shall serve as the core record.

The workflow becomes:

```
CUSTOMER ACCEPTANCE
 ↓
PROFESSIONAL REVIEW
 ↓
COMMENTS / REDLINES
 ↓
REVISION
 ↓
PROFESSIONAL REVIEW
 ↓
REVIEW RESULT
```

---

43. PROFESSIONAL REVIEW SCOPE

A professional review shall identify:

```
reviewer_id
professional_role
review_scope
revision_id
review_package_id
result
comments
reviewed_at
```

The system shall not infer professional authority merely from an account role.

---

44. PROFESSIONAL REVIEW RESULTS

Initial results:

```
APPROVED
APPROVED_WITH_COMMENTS
CHANGES_REQUIRED
NOT_APPROVED
WITHDRAWN
```

---

45. PROFESSIONAL REVIEW GATE

Where a project workflow requires professional review, finalization shall not occur until the required professional review state has been satisfied.

The requirement is project-specific and configurable.

---

46. MUNICIPAL APPROVAL

Municipal approval is an external authority process.

OneDraft shall track the process without representing itself as the municipal authority.

The existing:

```
projects.approval_statuses
```

table shall provide the base record.

---

47. MUNICIPAL APPROVAL RECORD

The approval record may contain:

```
authority
application_reference
status
submitted_at
decision_at
notes
```

Additional metadata may include:

```
submission_package_id
response_reference
decision_document_reference
```

---

48. APPROVAL STATES

Initial states:

```
NOT_SUBMITTED
PREPARING
SUBMITTED
UNDER_REVIEW
REVISION_REQUESTED
APPROVED
APPROVED_WITH_CONDITIONS
REJECTED
WITHDRAWN
EXPIRED
```

---

49. EXTERNAL AUTHORITY BOUNDARY

OneDraft shall never manufacture municipal approval.

A project may only be marked:

```
APPROVED
```

through an authorized workflow based on recorded external approval information.

---

50. SUBMISSION PACKAGE

A municipal submission package shall identify:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
document_package_id
submission_timestamp
authority
application_reference
package_hash
```

This creates a reproducible submission record.

---

51. APPROVAL DOCUMENTS

External approval documents may be associated with the submission.

They shall be stored through the existing document/object-storage subsystem.

The database stores their:

```
reference
hash
metadata
source
timestamp
```

rather than assuming the relational database itself is the binary document store.

---

52. REVIEW GATES

OneDraft shall implement explicit workflow gates.

Initial gates:

```
MODEL_READY
DOCUMENTATION_READY
INTERNAL_REVIEW_READY
CUSTOMER_REVIEW_READY
CUSTOMER_ACCEPTED
PROFESSIONAL_REVIEW_READY
PROFESSIONALLY_REVIEWED
SUBMISSION_READY
SUBMITTED
APPROVED
FINALIZATION_READY
FINALIZED
```

---

53. GATE EVALUATION

A gate is satisfied only when its required conditions are satisfied.

Example:

```
CUSTOMER_REVIEW_READY
```

may require:

```
document package exists
validation completed
no blocking validation failures
review package hash exists
revision is current
```

---

54. BLOCKING CONDITIONS

A workflow gate may be blocked by:

```
unresolved critical redlines
failed mandatory validation
missing required documents
missing required acceptance
missing professional review
revision mismatch
invalid code manifest
incomplete package
```

---

55. WORKFLOW ENGINE

The subsystem shall represent workflow state explicitly rather than relying on implicit UI state.

Conceptually:

```
current_state
required_conditions
completed_conditions
blocking_conditions
next_states
```

---

56. PROJECT REVIEW STATUS

The project shall expose a derived review status such as:

```
DRAFT
```

```
INTERNAL_REVIEW
CUSTOMER_REVIEW
REVISION_REQUIRED
PROFESSIONAL_REVIEW
SUBMISSION_READY
SUBMITTED
APPROVAL_PENDING
APPROVED
FINALIZATION_READY
FINALIZED
```

This status is derived from underlying records and workflow conditions.

---

57. STATUS DERIVATION

The review status shall not become an independent source of truth.

It is calculated from:

```
current_revision
review_sessions
review_items
redlines
validation_runs
customer_acceptance
professional_reviews
approval_statuses
finalization_state
```

---

58. FINALIZATION

Finalization creates the controlled endpoint of the OneDraft production workflow.

A project becomes finalizable only when all required gates have been satisfied.

---

59. FINALIZATION REQUIREMENTS

The default finalization requirements include:

```
current revision identified
model version identified
code manifest identified
required validation completed
blocking validation issues resolved
required documentation generated
document package generated
required review completed
```

```
required customer acceptance completed
required professional review completed
```

Municipal approval is included where the project workflow requires it.

---

60. FINALIZATION RECORD

`revisions.finalization_records`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
document_package_id UUID NOT NULL
code_manifest_id UUID
validation_run_id UUID
finalized_by UUID NOT NULL
finalized_at TIMESTAMPTZ NOT NULL
package_hash VARCHAR(128) NOT NULL
status VARCHAR(32) NOT NULL
notes TEXT
```

---

61. FINALIZATION STATES

```
READY
FINALIZED
REOPENED
SUPERSEDED
```

A finalized revision is immutable as a historical record.

---

62. REOPENING A FINALIZED PROJECT

A finalized project shall not be silently edited.

A new revision must be created.

Conceptually:

```
FINALIZED REVISION
 ↓
REOPEN REQUEST
 ↓
NEW REVISION
 ↓
```

```
MODEL CHANGE
 ↓
VALIDATION
 ↓
DOCUMENTATION
 ↓
REVIEW
```

The historical finalized state remains preserved.

---

63. FINAL DOCUMENT PACKAGE

The final document package shall identify:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
review_package_id
acceptance_record_id
professional_review_id
package_hash
```

This establishes the complete provenance chain.

---

64. PACKAGE HASHING

The final package shall be hashed.

The hash shall represent the exact finalized artifact set.

Conceptually:

```
documents
 ↓
canonical ordering
 ↓
package creation
 ↓
cryptographic hash
 ↓
finalization record
```

---

65. PROVENANCE CHAIN

The final project record shall permit reconstruction of:

```
CUSTOMER REQUIREMENTS
 ↓
PROJECT MODEL
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
 ↓
VALIDATION
 ↓
DRAWINGS
 ↓
REDLINES
 ↓
REVISED MODEL
 ↓
REVISED DOCUMENTATION
 ↓
CUSTOMER ACCEPTANCE
 ↓
PROFESSIONAL REVIEW
 ↓
MUNICIPAL SUBMISSION
 ↓
APPROVAL
 ↓
FINAL PACKAGE
```

---

66. AUDIT EVENTS

Every significant review action shall generate an audit event through:

`audit.project_events`

Examples:

```
REVIEW_CREATED
REVIEW_STARTED
REVIEW_COMPLETED
REVIEW_ITEM_CREATED
REDLINE_CREATED
REDLINE_ACCEPTED
REDLINE_IMPLEMENTED
REDLINE_RESOLVED
CHANGE_REQUEST_CREATED
CHANGE_REQUEST_COMPLETED
CUSTOMER_REVIEW_ISSUED
CUSTOMER_ACCEPTED
PROFESSIONAL_REVIEW_COMPLETED
SUBMISSION_CREATED
APPROVAL_RECORDED
PROJECT_FINALIZED
PROJECT_REOPENED
```

---

67. AUDIT IMMUTABILITY

Review and acceptance events are append-only.

Corrections shall create additional events.

Historical events shall not be silently overwritten.

---

68. REVIEWER IDENTITY

Every review action shall identify the acting principal.

The actor may be:

```
user
service
Aria
system
external authority record
```

where applicable.

The actor identity shall be traceable through the existing identity architecture.

---

69. AI / HUMAN SEPARATION

OneDraft shall distinguish:

AI-generated proposal

from:

human review

from:

human acceptance

from:

professional determination

from:

external authority decision

The system shall never collapse these into a single "approved by AI" state.

---

70. ARIA REVIEW ASSISTANT

Aria may assist reviewers by identifying:

```
potential inconsistencies
duplicate findings
affected objects
affected drawings
likely dependencies
validation requirements
documentation impacts
```

Aria may recommend resolutions.

The reviewer remains the controlling actor for review decisions assigned to a human reviewer.

---

71. REDLINE PRIORITIZATION

Aria may prioritize redlines using:

```
severity
dependency count
validation impact
documentation impact
project phase
deadline
review priority
```

The resulting priority is advisory unless explicitly configured as an automated workflow rule.

---

72. AUTOMATED REDLINE CHECK

Before marking a redline resolved, OneDraft shall verify:

```
target still exists
target revision is current
requested operation completed
affected geometry valid
affected documentation regenerated
required validation completed
```

If any required condition fails:

```
resolution = BLOCKED
```

---

73. REVIEW COMPARISON

The system shall support comparison between:

```
Revision N
```

and:

```
Revision N+1
```

Comparison may identify:

```
added objects
removed objects
modified objects
moved objects
changed parameters
changed constraints
changed drawings
changed schedules
changed annotations
```

---

74. DOCUMENT COMPARISON

The Documentation Engine may produce visual or semantic document comparisons.

The Review Engine consumes those comparisons as review evidence.

The Review Engine does not become responsible for geometric rendering.

---

75. REVIEW DASHBOARD API

Initial endpoint:

```
GET /api/v1/projects/{project_id}/review
```

Response:

```
{
  "data": {
    "project_id": "uuid",
    "revision_id": "uuid",
    "status": "CUSTOMER_REVIEW",
    "open_items": 4,
    "open_redlines": 2,
    "blocking_items": 0,
    "validation_status": "PASSED",
    "acceptance_status": "PENDING",
    "professional_review_status": "NOT_STARTED",
```

```
    "approval_status": "NOT_SUBMITTED"
  }
}
```

---

76. REVIEW SESSION API

Create:

```
POST /api/v1/projects/{project_id}/reviews
```

Request:

```
{
  "revision_id": "uuid",
  "review_type": "CUSTOMER_REVIEW"
}
```

Response:

```
{
  "data": {
    "review_session_id": "uuid",
    "status": "CREATED"
  }
}
```

---

77. REVIEW ITEM API

Create:

```
POST /api/v1/projects/{project_id}/reviews/{review_id}/items
```

Request:

```
{
  "drawing_id": "uuid",
  "target_object_id": "uuid",
  "category": "DESIGN",
  "severity": "MAJOR",
  "description": "Rear bedroom wall requires relocation."
}
```

---

78. REDLINE API

Create:

```
POST /api/v1/projects/{project_id}/redlines
```

Request:

```
{
  "review_item_id": "uuid",
  "drawing_id": "uuid",
  "target_object_id": "uuid",
  "instruction": "Move the rear bedroom wall 300 mm north.",
  "priority": "HIGH",
  "markup_geometry": {}
}
```

Response:

```
{
  "data": {
    "redline_id": "uuid",
    "status": "OPEN"
  }
}
```

---

79. REDLINE RESOLUTION API

Resolve:

```
POST /api/v1/projects/{project_id}/redlines/{redline_id}/resolve
```

The server shall first verify that the underlying change has been implemented and validated.

The request cannot independently declare a model change complete.

---

80. CUSTOMER REVIEW API

Issue package:

```
POST /api/v1/projects/{project_id}/customer-review
```

Request:

```
{
  "revision_id": "uuid",
  "document_package_id": "uuid",
  "customer_id": "uuid"
}
```

---

81. CUSTOMER ACCEPTANCE API

Accept:

```
POST /api/v1/projects/{project_id}/acceptance
```

Request:

```
{
  "revision_id": "uuid",
  "document_package_id": "uuid",
  "acknowledgements": {
    "reviewed_revision": true,
    "reviewed_documents": true
  }
}
```

The server verifies that the package corresponds to the identified revision.

---

82. PROFESSIONAL REVIEW API

Create:

```
POST /api/v1/projects/{project_id}/professional-review
```

Request:

```
{
  "revision_id": "uuid",
  "review_package_id": "uuid",
  "reviewer_id": "uuid",
  "professional_role": "ARCHITECT",
  "review_scope": "Architectural documentation review"
}
```

---

83. APPROVAL API

Create submission:

```
POST /api/v1/projects/{project_id}/approvals
```

Request:

```
{
  "revision_id": "uuid",
  "document_package_id": "uuid",
  "authority": "Municipal Authority",
  "application_reference": "..."
}
```

---

84. FINALIZATION API

Check readiness:

```
GET /api/v1/projects/{project_id}/finalization/readiness
```

Response:

```
{
  "data": {
    "ready": true,
    "revision_id": "uuid",
    "conditions": {
      "validation": "PASSED",
      "documentation": "COMPLETE",
      "customer_acceptance": "ACCEPTED",
      "professional_review": "APPROVED",
      "approval": "APPROVED"
    }
  }
}
```

---

85. FINALIZE PROJECT

Endpoint:

```
POST /api/v1/projects/{project_id}/finalize
```

Request:

```
{
  "revision_id": "uuid"
}
```

The service performs a final gate evaluation before creating the finalization record.

---

86. FINALIZATION TRANSACTION

Conceptually:

```
BEGIN

verify revision
verify model version
verify code manifest
verify validation
verify documentation
```

```
verify review
verify acceptance
verify professional review
verify required approval
verify package integrity

create finalization record
record audit event

COMMIT
```

If any mandatory condition fails:

```
ROLLBACK
```

---

87. PROJECT CLOSURE

Once finalized, the project enters:

```
FINALIZED
```

The project remains queryable.

Historical records remain available.

The project is not physically deleted.

---

88. ARCHIVAL

Future archival may transition a finalized project to:

```
ARCHIVED
```

Archival shall preserve:

```
model
revisions
validation
documentation
review
acceptance
approval
audit
```

---

89. RECONSTRUCTION REQUIREMENT

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
Finalization_ID
```

OneDraft must be able to reconstruct:

```
project state
review state
validation state
document package
acceptance state
approval state
audit history
```

---

90. FINALIZATION INTEGRITY TEST

The finalization subsystem shall verify:

```
project identity
revision identity
model version identity
code manifest identity
validation identity
document package identity
review package identity
acceptance identity
professional review identity
approval identity
package hash
```

before finalization.

---

91. REVIEW REGRESSION TEST

The first major Review Engine integration test shall execute:

```
1. Create Revision 1

2. Generate documentation

3. Create internal review

4. Create review findings
```

5. Create redlines

6. Submit redlines for implementation

7. Generate Aria command proposals

8. Execute approved commands

9. Create Revision 2

10. Validate Revision 2

11. Regenerate affected documentation

12. Create Revision 2 review package

13. Complete internal review

14. Issue customer review

15. Record customer changes

16. Resolve customer redlines

17. Generate Revision 3

18. Validate Revision 3

19. Regenerate documentation

20. Record customer acceptance

21. Create professional review

22. Record professional result

23. Create approval submission

24. Record approval

25. Verify finalization readiness

26. Finalize project

27. Hash final package

28. Retrieve complete audit history

---

92. REVIEW ENGINE SUCCESS CRITERIA

The subsystem is successful when it can:

Create a review session

Review a specific revision

Create review findings

Create redlines

Route redlines to Aria

Create model commands

Create new revisions

Regenerate affected documentation

Validate resulting changes

Resolve redlines

Issue customer review packages

Record customer acceptance

Record professional review

Track external approval

Determine finalization readiness

Finalize a project

Preserve complete review provenance

---

93. REVIEW STATE MACHINE

The complete default workflow becomes:

```
DRAFT
  ↓
MODEL_READY
  ↓
DOCUMENTATION_READY
  ↓
INTERNAL_REVIEW
  ↓
CHANGES_REQUIRED
  ↓
REVISION
  ↓
VALIDATION
  ↓
DOCUMENTATION_REGENERATION
  ↓
INTERNAL_REVIEW
  ↓
CUSTOMER_REVIEW
  ↓
CUSTOMER_CHANGES
```

```
  ↓
REVISION
  ↓
VALIDATION
  ↓
CUSTOMER_ACCEPTANCE
  ↓
PROFESSIONAL_REVIEW
  ↓
SUBMISSION
  ↓
APPROVAL
  ↓
FINALIZATION
  ↓
FINALIZED
```

The workflow may return to an earlier state whenever a required change is identified.

---

94. NON-LINEAR REVISION CONTROL

The system shall support repeated review cycles.

For example:

```
Revision 1
 ↓
Review
 ↓
Revision 2
 ↓
Review
 ↓
Revision 3
 ↓
Review
 ↓
Revision 4
```

Every revision remains independently reconstructable.

---

95. NO DESTRUCTIVE REVIEW

Review actions shall never overwrite the project state under review.

Instead:

```
OLD REVISION
+
REVIEW FINDINGS
+
```

```
COMMANDS
=
NEW REVISION
```

This preserves the historical project record.

---

96. REVIEW AND COMPLIANCE COORDINATION

When a review change affects a regulated condition, the Review Engine shall trigger the existing Compliance and Validation systems.

Conceptually:

```
REDLINE
 ↓
MODEL CHANGE
 ↓
COMPLIANCE IMPACT ANALYSIS
 ↓
VALIDATION
 ↓
DOCUMENTATION
```

The Review Engine coordinates the workflow but does not duplicate regulatory logic.

---

97. REVIEW AND GEOMETRY COORDINATION

When a review change affects geometry:

```
REDLINE
 ↓
MODEL COMMAND
 ↓
GEOMETRY ENGINE
 ↓
GEOMETRY VALIDATION
 ↓
DEPENDENCY UPDATE
 ↓
DOCUMENTATION
```

The Review Engine does not perform geometry computation itself.

---

98. REVIEW AND DOCUMENTATION COORDINATION

When a review change affects drawings:

```
MODEL REVISION
 ↓
DOCUMENT IMPACT ANALYSIS
 ↓
AFFECTED VIEWS
 ↓
AFFECTED DRAWINGS
 ↓
AFFECTED SHEETS
 ↓
DOCUMENT PACKAGE
```

---

99. REVIEW NOTIFICATION EVENTS

The subsystem shall generate application events for:

```
REVIEW_ASSIGNED
REVIEW_READY
REDLINE_CREATED
CHANGE_REQUEST_CREATED
CHANGE_REQUEST_COMPLETED
VALIDATION_REQUIRED
DOCUMENTATION_REGENERATION_REQUIRED
CUSTOMER_REVIEW_READY
CUSTOMER_ACTION_REQUIRED
PROFESSIONAL_REVIEW_REQUIRED
APPROVAL_ACTION_REQUIRED
FINALIZATION_READY
```

Notification delivery remains an application/infrastructure concern.

---

100. EVENT-DRIVEN ARCHITECTURE

The Review Engine shall communicate with other subsystems through domain events where appropriate.

Conceptually:

```
DocumentationGenerated
        ↓
ReviewReady

RedlineCreated
```

```
        ↓
ChangeRequested

ModelRevisionCreated
        ↓
ValidationRequired

ValidationCompleted
        ↓
DocumentationRegenerationRequired

CustomerAccepted
        ↓
ProfessionalReviewRequired

ApprovalRecorded
        ↓
FinalizationReadinessCheck
```

---

101. IDEMPOTENCY

All mutating review endpoints shall support:

```
Idempotency-Key
```

Repeated requests using the same valid idempotency key shall not create duplicate:

```
review sessions
redlines
acceptance records
approval submissions
finalization records
```

---

102. CONCURRENCY CONTROL

Review operations shall use optimistic concurrency.

Every review operation affecting a revision shall identify the expected revision.

Example:

```
expected_revision = 17
```

If the project is already at Revision 18:

```
409 REVISION_CONFLICT
```

The reviewer must review the current state before submitting additional changes.

---

103. REVIEW PACKAGE VERSION CONFLICT

If a reviewer submits comments against an obsolete package:

```
review_package_revision != current_revision
```

the system shall identify the package as stale.

The original review remains historically valid.

The new current revision requires a new review package.

---

104. SECURITY BOUNDARY

Review permissions shall require both:

authorization scope

and:

project membership

Examples:

```
review:read
review:write
redline:read
redline:write
acceptance:write
professional_review:write
approval:write
finalize:write
```

---

105. CUSTOMER ACCESS BOUNDARY

Customer users shall only access review packages explicitly issued to them.

A customer shall not automatically receive:

```
internal review notes
internal system diagnostics
AI reasoning artifacts
restricted compliance data
professional internal comments
```

unless explicitly exposed through the application contract.

---

106. PROFESSIONAL ACCESS BOUNDARY

Professional reviewers shall receive only the project and review scope authorized for their assignment.

Professional review records shall remain distinct from customer comments and internal AI processing.

---

107. FINAL PACKAGE ACCESS

After finalization, authorized users may retrieve:

```
final document package
revision metadata
validation summary
code manifest metadata
acceptance record
professional review record
approval status
```

subject to access permissions.

---

108. FINAL PACKAGE PROVENANCE

The final package shall provide a machine-readable provenance manifest.

Conceptually:

```
{
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "validation_run_id": "uuid",
  "review_package_id": "uuid",
  "acceptance_record_id": "uuid",
  "professional_review_id": "uuid",
  "approval_record_id": "uuid",
  "package_hash": "..."
}
```

---

109. FINAL AUDIT RECORD

Finalization shall produce an audit event containing:

```
project_id
revision_id
model_version_id
```

```
finalization_id
package_hash
actor_id
timestamp
```

The event becomes part of the permanent project history.

---

110. REVIEW ENGINE DATABASE MIGRATION ORDER

The next migration sequence shall extend the existing migration architecture:

```
026_review_sessions
027_review_items
028_change_requests
029_review_packages
030_customer_reviews
031_finalization_records
032_review_indexes
033_review_events
034_review_seed_data
```

Existing tables such as:

```
revisions.redlines
revisions.redline_resolutions
revisions.acceptance_records
revisions.professional_reviews
projects.approval_statuses
audit.project_events
```

shall be extended through controlled migrations rather than duplicated.

---

111. OPENAPI RESOURCE GROUPS

The Review API shall expose:

```
Reviews
Review Items
Redlines
Change Requests
Review Packages
Customer Reviews
Acceptance
Professional Reviews
Approvals
Finalization
```

---

112. API CONTRACT

Representative endpoints:

```
GET    /api/v1/projects/{project_id}/review

POST   /api/v1/projects/{project_id}/reviews

GET    /api/v1/projects/{project_id}/reviews/{review_id}

POST   /api/v1/projects/{project_id}/reviews/{review_id}/items

POST   /api/v1/projects/{project_id}/redlines

GET    /api/v1/projects/{project_id}/redlines

POST   /api/v1/projects/{project_id}/redlines/{redline_id}/resolve

POST   /api/v1/projects/{project_id}/change-requests

GET    /api/v1/projects/{project_id}/review-packages

POST   /api/v1/projects/{project_id}/customer-review

POST   /api/v1/projects/{project_id}/acceptance

POST   /api/v1/projects/{project_id}/professional-review

POST   /api/v1/projects/{project_id}/approvals

GET    /api/v1/projects/{project_id}/finalization/readiness

POST   /api/v1/projects/{project_id}/finalize
```

---

113. REVIEW SERVICE BOUNDARY

The implementation shall establish:

```
project_review_service.py
```

as the primary application service boundary.

Supporting components may include:

```
review_session_service.py
redline_service.py
change_request_service.py
acceptance_service.py
professional_review_service.py
approval_service.py
finalization_service.py
```

---

114. DOMAIN TYPES

The Python domain layer shall define types for:

```
ReviewSession
ReviewItem
ReviewDisposition
Redline
RedlineResolution
ChangeRequest
ReviewPackage
CustomerReview
AcceptanceRecord
ProfessionalReview
ApprovalStatus
FinalizationRecord
FinalizationReadiness
```

---

115. FINALIZATION SERVICE

The core service shall conceptually implement:

```
check_finalization_readiness(project_id, revision_id)
```

and:

```
finalize_project(project_id, revision_id)
```

The second operation shall invoke the first before modifying persistent state.

---

116. REVIEW TRANSACTION BOUNDARY

A review operation that creates a persistent review state shall execute atomically.

Conceptually:

```
BEGIN

verify authorization
verify project
verify revision
create review record
create audit event

COMMIT
```

---

117. REDLINE TRANSACTION BOUNDARY

A redline implementation shall never be recorded as complete before the underlying model operation is committed.

The sequence is:

```
BEGIN

verify redline
verify revision
execute command
create revision
record change
record redline implementation
record audit event

COMMIT
```

If the model mutation fails:

```
ROLLBACK
```

and the redline remains unresolved.

---

118. ACCEPTANCE TRANSACTION BOUNDARY

Acceptance shall execute:

```
BEGIN

verify customer authorization
verify package
verify revision
verify package hash
verify review requirements
create acceptance record
record audit event

COMMIT
```

---

119. FINALIZATION TRANSACTION BOUNDARY

Finalization shall execute:

```
BEGIN

verify all mandatory gates
verify package
verify hashes
verify revision
create finalization record
lock final package
record finalization event

COMMIT
```

No incomplete finalization state may be exposed as finalized.

---

120. REVIEW ENGINE SUCCESS TEST

The complete subsystem is successful when OneDraft can execute:

```
PROJECT
 ↓
REVISION
 ↓
DOCUMENT PACKAGE
 ↓
REVIEW
 ↓
REDLINE
 ↓
ARIA COMMAND
 ↓
MODEL REVISION
 ↓
GEOMETRY UPDATE
 ↓
COMPLIANCE VALIDATION
 ↓
DOCUMENT REGENERATION
 ↓
REVIEW
 ↓
CUSTOMER ACCEPTANCE
 ↓
PROFESSIONAL REVIEW
 ↓
MUNICIPAL SUBMISSION
 ↓
APPROVAL
 ↓
FINALIZATION
```

while preserving every intermediate state.

---

121. COMPLETE ONEDRAFT PRODUCTION PIPELINE

With this subsystem established, the OneDraft architecture now forms the complete production chain:

```
PROJECT REQUIREMENTS
        ↓
COMPUTATIONAL PROJECT MODEL
        ↓
PARAMETRIC GEOMETRY
        ↓
REGULATORY ANALYSIS
        ↓
VALIDATION
        ↓
DOCUMENTATION
        ↓
REVIEW
        ↓
REDLINES
        ↓
MODEL REVISION
        ↓
VALIDATION
        ↓
DOCUMENT REGENERATION
        ↓
CUSTOMER REVIEW
        ↓
CUSTOMER ACCEPTANCE
        ↓
PROFESSIONAL REVIEW
        ↓
MUNICIPAL APPROVAL
        ↓
FINAL DOCUMENT PACKAGE
        ↓
FINALIZATION
        ↓
ARCHIVAL
```

---

122. ARCHITECTURAL BOUNDARIES NOW ESTABLISHED

OneDraft now has explicit boundaries for:

Project Persistence

API

Computational Project Model

Parametric Geometry

Compliance & Regulatory Analysis

Validation

Revision Control

Documentation & Drawing Generation

Review

Redline Management

Customer Acceptance

Professional Review

Municipal Approval Tracking

Finalization

Audit

These systems cooperate through controlled interfaces rather than directly modifying one another's internal state.

---

123. SYSTEM OF RECORD HIERARCHY

The complete hierarchy is now:

1. Project Model

2. Model Versions

3. Revisions

4. Regulatory Manifest

5. Validation Results

6. Generated Documentation

7. Review Records

8. Customer Acceptance

9. Professional Review

10. Municipal Approval Records

11. Finalization Record

12. Audit Events

The hierarchy preserves the distinction between computational state, derived artifacts, human decisions, external decisions, and historical evidence.

---

124. RECONSTRUCTION STANDARD

A finalized OneDraft project must be reconstructable from its persistent records.

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
Validation_Run_ID
Document_Package_ID
Finalization_ID
```

the system shall be capable of reconstructing:

```
project state
model state
geometry references
regulatory basis
validation state
documentation state
review history
acceptance history
professional review
approval history
final package
```

---

125. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package for this subsystem shall consist of:

Artifact A

`026_review_engine.sql`

The PostgreSQL migration extending the existing review, redline, acceptance, professional review, approval, and finalization structures.

Artifact B

`review_openapi.yaml`

The OpenAPI contract for review, redline, acceptance, approval, and finalization operations.

Artifact C

`review_types.py`

Python/Pydantic domain models for the complete review lifecycle.

Artifact D

`project_review_service.py`

The transactional service boundary coordinating review, redline resolution, acceptance, approval tracking, and finalization.

Artifact E

`finalization_service.py`

The gate-evaluation and finalization implementation.

Artifact F

`review_integration_test.py`

The first automated end-to-end review-to-finalization regression test.

---

126. NEXT STACK POSITION

The Review, Redline, Acceptance & Approval Engine completes the controlled human/project lifecycle surrounding the computational model.

The architecture therefore progresses from:

```
PERSISTENCE/API
        ↓
PROJECT MODEL
        ↓
GEOMETRY
        ↓
COMPLIANCE
        ↓
VALIDATION
        ↓
DOCUMENTATION
        ↓
REVIEW / REDLINE
        ↓
ACCEPTANCE / APPROVAL
        ↓
FINALIZATION
```

The next implementation concern is no longer redefining the project model or its drawing-generation mechanisms.

The next layer is the **OneDraft Execution, Orchestration & Job Infrastructure**, which connects the completed domain engines into an asynchronous production system capable of queuing, scheduling, executing, monitoring, retrying, cancelling, and completing long-running OneDraft operations.

---

127. SPECIFICATION STATUS

ONEDRAFT PROJECT REVIEW, REDLINE, ACCEPTANCE & APPROVAL ENGINE v0.1

STATUS: ESTABLISHED

The OneDraft project lifecycle is now architecturally defined from initial project state through controlled review and finalization.

The next stack is:

```
EXECUTION, ORCHESTRATION & JOB INFRASTRUCTURE
```

This subsystem will provide the runtime machinery connecting:

```
ARIA
```

```
PROJECT MODEL
```

```
GEOMETRY ENGINE
```

```
COMPLIANCE ENGINE
```

```
VALIDATION ENGINE
```

```
DOCUMENTATION ENGINE
```

```
REVIEW ENGINE
```

and:

```
FINALIZATION
```

into one executable production pipeline.